

Smart MCQ Solver Challenge

Ranking Answers for Multiple-Choice Questions with MAP@3

Deep Learning & Generative AI Project Report | July 2026

Student ID: 23f1002033 | Name: Ishank Gupta

Platform: Kaggle (GPU) | Tracking: Weights & Biases (project: 23f1002033-t22026)

1. Abstract / Executive Summary

This project tackles the Smart MCQ Solver Challenge: predict the top three most probable answers, out of five options (A-E), for challenging multiple-choice questions, evaluated by Mean Average Precision at 3 (MAP@3). Four modelling approaches were developed and compared: (i) a TF-IDF + Logistic Regression baseline, (ii) a fine-tuned DeBERTa-v3 transformer (5-fold CV, AutoModelForMultipleChoice), (iii) a from-scratch BiLSTM ranking model with a custom tokenizer, and (iv) a Retrieval-Augmented Generation (RAG) pipeline built from pretrained components (MPNet retriever, MiniLM cross-encoder reranker, DeBERTa-NLI reader). A final Distractor Ensemble - motivated by an EDA discovery that incorrect options carry a detectable writing style - formed the final submission. Because the dataset contains paraphrased duplicate questions, a random validation split leaks near-identical items and inflates validation MAP@3 toward 1.0; the Kaggle public leaderboard is therefore treated as the primary, trustworthy metric throughout. On the public leaderboard the BiLSTM scored 0.75050 (best single model), TF-IDF+LR 0.73940, RAG 0.74064, and the final Distractor Ensemble 0.74896. All experiments were tracked in Weights & Biases. The main learnings: strong sparse baselines set a high floor on small text datasets; validation design matters as much as model design when duplicates are present; and honest, leakage-aware evaluation - trusting the held-out leaderboard over an over-optimistic internal split - is essential to drawing correct conclusions.

2. Introduction

2.1 Problem Statement

The Kaggle competition provides 2,000 training questions and 500 test questions. Each row contains a question prompt, five candidate options (A-E), and (in training) the correct answer label. The task is to output three ranked labels per test question (e.g., "B A D"). The metric, MAP@3, awards 1.0 / 0.5 / 0.33 when the correct option is ranked 1st / 2nd / 3rd, and 0 otherwise - so the task is answer ranking, not plain classification.

2.2 Project Objective

- Build and compare at least three model families: a from-scratch deep model, a fine-tuned pretrained transformer, and one model of choice (RAG / ensemble).
- Directly optimise and honestly validate MAP@3 with a shared, leakage-free evaluation protocol.
- Master the project toolchain end-to-end: PyTorch, Hugging Face transformers/datasets, FAISS, sentence-transformers, and Weights & Biases experiment tracking.

2.3 Report Structure

Section 3 covers the dataset, EDA and preprocessing; Section 4 the tokenization strategies; Section 5 the models (fine-tuned transformer, custom model, RNN-based model, and additional models); Section 6 the comparative performance analysis with W&B evidence; Section 7 conclusions, challenges and future work; Section 8 references.

3. Dataset & Preprocessing

3.1 Dataset Description

Property	Value
Training rows	2,000 questions (id, prompt, A-E, answer)
Test rows	500 questions (no answer column)
Options per question	5 (A-E), exactly one correct
Output format	"ID, Prediction" where Prediction = 3 space-separated ranked labels
Domains	Mixed knowledge domains - philosophy, physics, astronomy, history, and more

3.2 Exploratory Data Analysis

Answer balance. The correct answer is not perfectly uniform: B (490) and C (460) occur noticeably more often than E (325), roughly a 1.5:1 ratio between the most and least frequent labels. This is mild imbalance rather than severe skew, so no class re-weighting was applied; macro-F1 is reported alongside accuracy to confirm no option label is systematically neglected. The models are also trained with a softmax over the five options within each question rather than as an independent global 5-class classifier, which further limits the influence of label frequency.

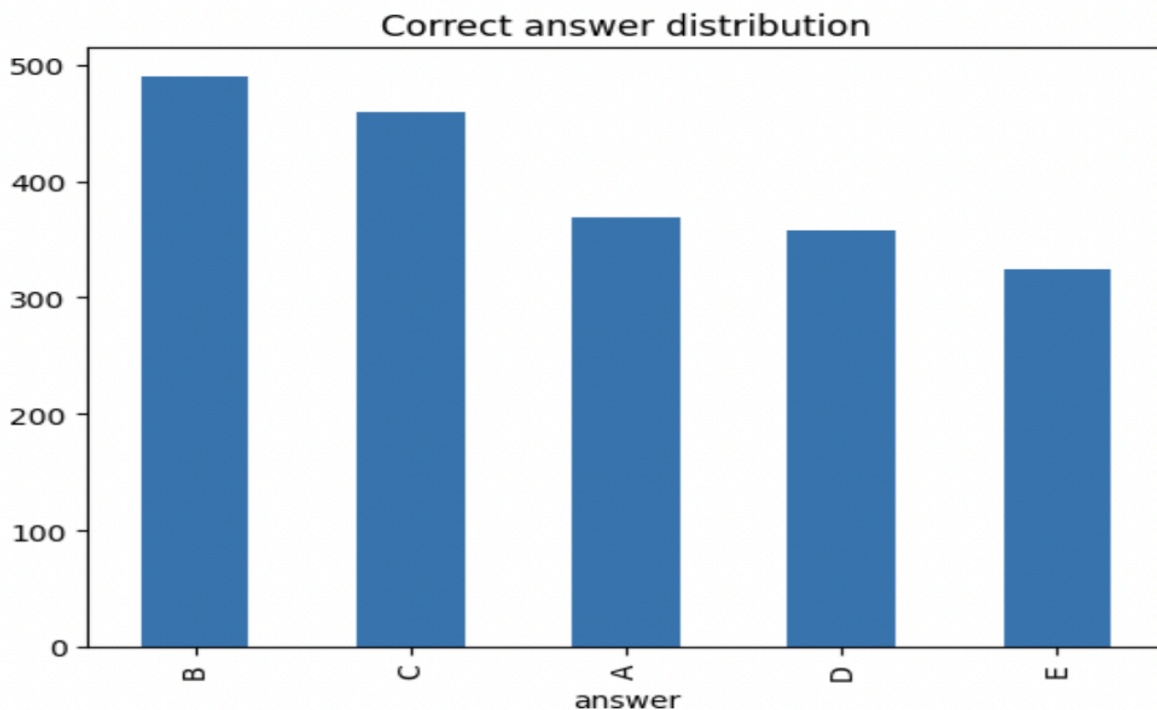


Figure 1: Correct-answer distribution across options A-E (training set).

Text lengths. Average prompt length is **18.1** words and options average **26.2** words, with some options running to full paragraphs. This motivated a 64-token cap for the scratch model and 256 tokens for the transformer.

Paraphrased duplicates. Many prompts wrap the same underlying question in different instruction templates ("Pick the best possible answer: ...", "Select the most accurate option: ...") with identical option sets. This near-duplication is a key dataset property: it makes retrieval-based approaches unusually effective, but also demands care to avoid optimistic validation.

Key finding - distractor style artifact. Treating every option as an independent text and predicting "is this the correct option?" with TF-IDF + Logistic Regression - without ever seeing the question - achieves AUC = **0.9789**. Incorrect options carry a detectable writing style (the most negative coefficients are hedging/absolute terms typical of fabricated distractors). This artifact directly motivated the Distractor Ensemble (Section 5.4).

3.3 Data Preprocessing

- No missing values in prompt or options; all fields cast to string defensively before tokenization.
- Answer labels mapped A-E -> 0-4 for model training.
- For sparse models, prompt and options are concatenated with explicit markers ("prompt A: ... B: ..."); milestone experiments used lowercasing + punctuation stripping for the cosine-similarity baselines.
- Shared stratified 90/10 train/validation split (seed 42, 200 held-out questions) so every model's validation MAP@3 is computed on identical data; the transformer additionally uses 5-fold stratified CV with out-of-fold (OOF) predictions.
- **Known limitation - validation leakage:** because paraphrased duplicates (Section 3.2) can place near-identical questions on both sides of a random split, internal validation MAP@3 is optimistically high (approaching 1.0). All headline conclusions therefore rely on the Kaggle public leaderboard, which is measured on genuinely unseen questions. A group-aware split keyed on question identity is proposed as future work (Section 7.3).

3.4 Data Augmentation

No synthetic text augmentation was applied - with only 2,000 rows and near-duplicate questions already present, paraphrase augmentation risked amplifying leakage. Instead, robustness came from model diversity (word vs character features, sequential vs retrieval models) and fold averaging.

4. Tokenization Strategy

4.1 Primary Tokenizer - Custom Word-Level (Best Single Model)

The best-performing leaderboard model (BiLSTM, 0.75050) uses a from-scratch tokenizer: lowercase regex tokens [a-z0-9]+, a vocabulary built from training text with minimum frequency 2, and <pad>/<unk> specials. Justification: the vocabulary is small and domain-closed (train and test share topics and even paraphrased questions), so a word-level vocabulary covers the test distribution well while keeping the embedding table tiny - appropriate for training from scratch on 1,800 examples. Sequences are truncated/padded to 64 tokens; padding positions are masked out of the mean-pooling so they never influence sentence vectors.

4.2 Transformer Tokenizer - DeBERTa-v3 SentencePiece

The fine-tuned transformer (Section 5.1) uses the microsoft/deberta-v3-base tokenizer - a SentencePiece Unigram subword tokenizer with a 128k vocabulary. Subword units eliminate the

out-of-vocabulary problem for rare technical terms (e.g., "accelerator-based", proper nouns), which is exactly where the word-level tokenizer degrades to <unk>. Each question is expanded into five (prompt, option) pairs tokenized jointly with truncation to 256 tokens; a custom multiple-choice collator pads dynamically per batch and reshapes to (batch, 5, seq_len), avoiding wasteful fixed-length padding.

4.3 Experimental Comparison

- **Word TF-IDF (1-2-grams) vs char TF-IDF (3-5-grams char_wb):** both used in the Distractor Ensemble; character n-grams proved robust to morphology and typos and complemented word features.
- **Word-level vocab (min-freq 2) for BiLSTM:** chosen over subword after observing the closed domain; the known trade-off - rare terms collapsing to <unk> - is analysed in Section 6.5.
- **SentencePiece (DeBERTa-v3) for fine-tuning:** required by the pretrained checkpoint; also used by the sentence-transformer retriever and NLI reader in the RAG pipeline.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer - DeBERTa-v3 (5-Fold CV)

Architecture

microsoft/deberta-v3-base (He et al., 2021; Hugging Face model card: huggingface.co/microsoft/deberta-v3-base) - a 12-layer, 768-hidden, 86M-backbone-parameter encoder. DeBERTa-v3 improves on BERT with disentangled attention (content and relative-position embeddings attend separately) and is pretrained with ELECTRA-style replaced-token detection with gradient-disentangled embedding sharing, giving it state-of-the-art quality per parameter among base-size encoders. It is wrapped in AutoModelForMultipleChoice: each of the five (prompt, option) pairs is encoded independently, a shared linear head maps each [CLS] representation to one scalar, and softmax over the five scalars produces option probabilities - an architecture that matches the ranking objective exactly.

Salient Points

- Disentangled self-attention lets the model weigh fine-grained factual differences between long, nearly identical options - the hardest question type in this dataset.
- The multiple-choice head compares options against each other through the shared softmax, rather than scoring them in isolation.

Fine-Tuning Strategy

Parameter	Value	Justification
Folds / epochs	5-fold Stratified CV, 3 epochs per fold	Small dataset; best epoch is restored via load_best_model_at_end
Optimiser / LR	AdamW, lr = 5e-6, weight decay 0.01	2e-5 caused collapse; 5e-6 is the stable band for DeBERTa-v3
Scheduler	Cosine decay with 15% warmup	Longer warmup prevents the early divergence seen in the failed run
Batch / max length	4 x 5 options, grad-accum 4 (eff. batch 16) / 256 tokens	Larger effective batch smooths noisy gradients on a small dataset
Precision	fp32	bf16 contributed to exploding loss; fp32 is numerically safer here

Gradient clipping	max-norm 0.3	Caps the exploding gradients observed in the failed run
Checkpoint selection	load_best_model_at_end (metric = MAP@3)	Keeps the best epoch even if later epochs degrade
Layers frozen	None (full fine-tuning)	Low LR plus clipping is gentle enough; LoRA variant explored in Milestone 4

Out-of-fold CV MAP@3 = **0.9928 (5-fold OOF; per-fold 0.995 / 0.981 / 0.995 / 0.993 / 1.000)** (per-fold scores and cv_std logged to W&B run deberta-v3-5fold-cv). Test predictions average the logits of all five fold models. An initial run at learning rate 2e-5 for 2 epochs collapsed to near-random (OOF MAP@3 ~ 0.38). The diagnosis came from two signals: training loss spiked to values of 12 to 31 against an expected $\ln(5)$ of about 1.6, and validation loss froze at an identical value across consecutive epochs. Together these indicate exploding gradients followed by output collapse, a known DeBERTa-v3 fine-tuning instability in which the model degenerates into predicting nearly the same option for every question. Four changes fixed it: the learning rate was lowered to 5e-6, precision was switched from bf16 to fp32, gradient clipping was tightened to max-norm 0.3, and load_best_model_at_end was enabled so the best-scoring epoch is restored even if later epochs degrade. All five folds then trained cleanly. Parameter-efficient fine-tuning (LoRA via PEFT) was also practised in Milestone 4 on the same multiple-choice formulation, adapting only low-rank matrices injected into attention projections.

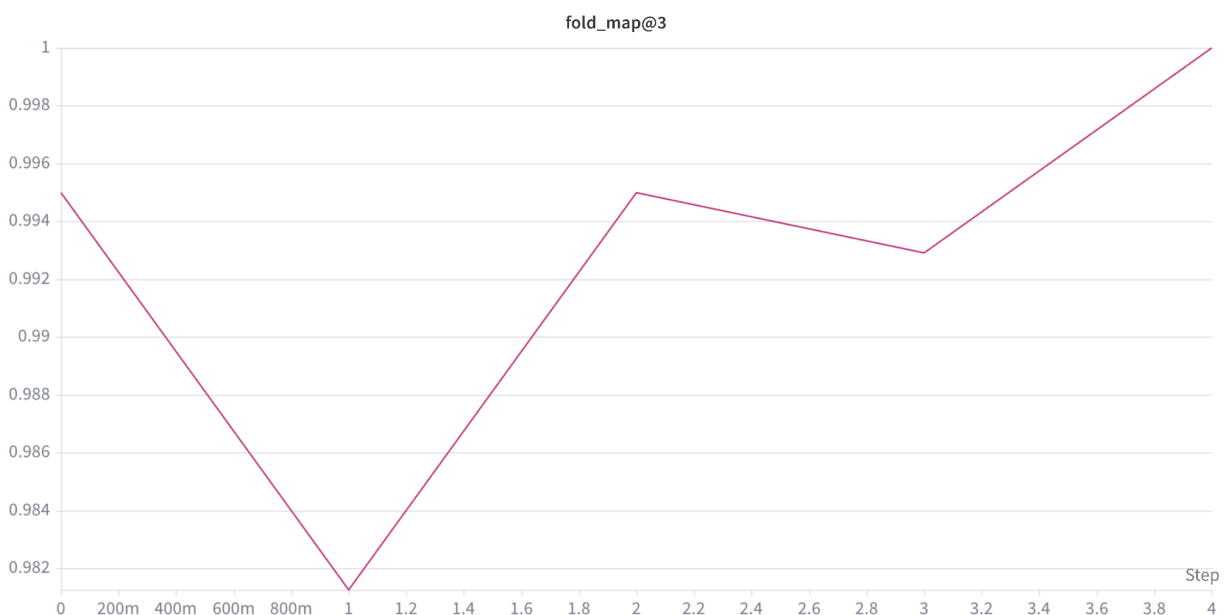


Figure 2: DeBERTa-v3 5-fold fine-tuning - evaluation MAP@3 (W&B run: deberta-v3-5fold-cv).

5.2 Model 2: Custom Deep Learning Model - BiLSTM Matcher (From Scratch)

Architecture

The custom model is a siamese-style ranking network built entirely from scratch - tokenizer, vocabulary, embeddings and all. A shared encoder (Embedding -> BiLSTM -> masked mean-pooling) maps the prompt and each of the five options to 256-d vectors. For every option, an interaction feature vector $[p ; o ; |p-o| ; p*o]$ (1024-d) feeds a two-layer scorer producing one scalar; the five scalars are the logits of a softmax trained with cross-entropy. The architecture diagram is shown below.

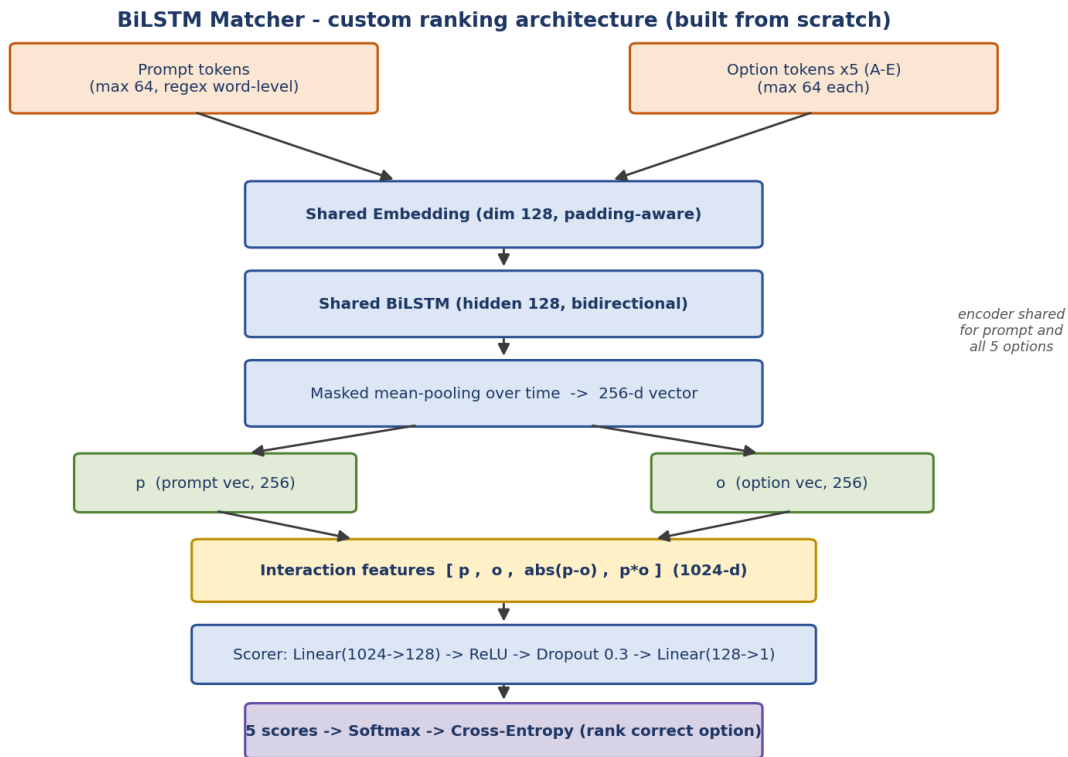


Figure 3: BiLSTM Matcher architecture - shared encoder, interaction features, and option scorer.

Salient Points

- Weight sharing between prompt and option encoders places both in the same embedding space, so the interaction features are directly comparable - and cuts parameters, critical with 1,800 training rows.
- The interaction vector is deliberately richer than cosine similarity: $|p-o|$ captures element-wise disagreement and $p*o$ captures agreement, letting the scorer learn which dimensions matter.
- Training with softmax over the five options optimises exactly what MAP@3 rewards - ranking the correct option above its four distractors - rather than independent binary decisions.

Hyperparameter	Value
Embedding / hidden dim	128 / 128 (BiLSTM output 256)
Max length / vocab	64 tokens; min-freq-2 vocabulary
Batch / epochs	32 / 10
Optimiser / scheduler	Adam lr 1e-3; CosineAnnealingLR over all steps
Regularisation	Dropout 0.3 in scorer; padding-masked pooling
Loss	CrossEntropyLoss over 5 option scores

5.3 Model 3: RNN-Based Model - the Bidirectional LSTM Encoder

Architecture

The recurrent core of the custom model is a single-layer bidirectional LSTM (hidden 128 per direction). Token embeddings are processed left-to-right and right-to-left; the concatenated 256-d hidden states are mean-pooled over non-padding positions to form the sentence vector (see Figure 3).

Salient Points - why a sequence model, and these choices

- Word order matters here: options often differ only by a swapped pair of terms ("light-ion fusion" vs "heavy-ion fusion", "10 kV" vs "100 kV"). Bag-of-words models cannot see which noun a modifier attaches to; an LSTM reads the sequence.
- Bidirectionality: an answer's decisive phrase can sit anywhere in a paragraph-length option; the backward pass gives every position right-context as well as left-context. LSTM was preferred over GRU for its separate cell state, which preserves information across these long options.
- Masked mean-pooling over the final hidden state: with up-to-64-token padded sequences, the last hidden state over-weights sequence tails and padding; averaging all valid positions is a more faithful summary and trained noticeably more stably.
- One layer only: deeper stacks overfit immediately on 1,800 examples; capacity was spent on the interaction scorer instead.

Best validation MAP@3 = 0.7867 (validation; see leakage note in Section 6.3) - leaderboard 0.75050, the best single model; the public leaderboard score of 0.75050 makes it the best single model in the project. Step-level training loss, learning-rate schedule and epoch-level validation MAP@3 are logged in W&B runs `bilstm-from-scratch` / `final-bilstm-training`.

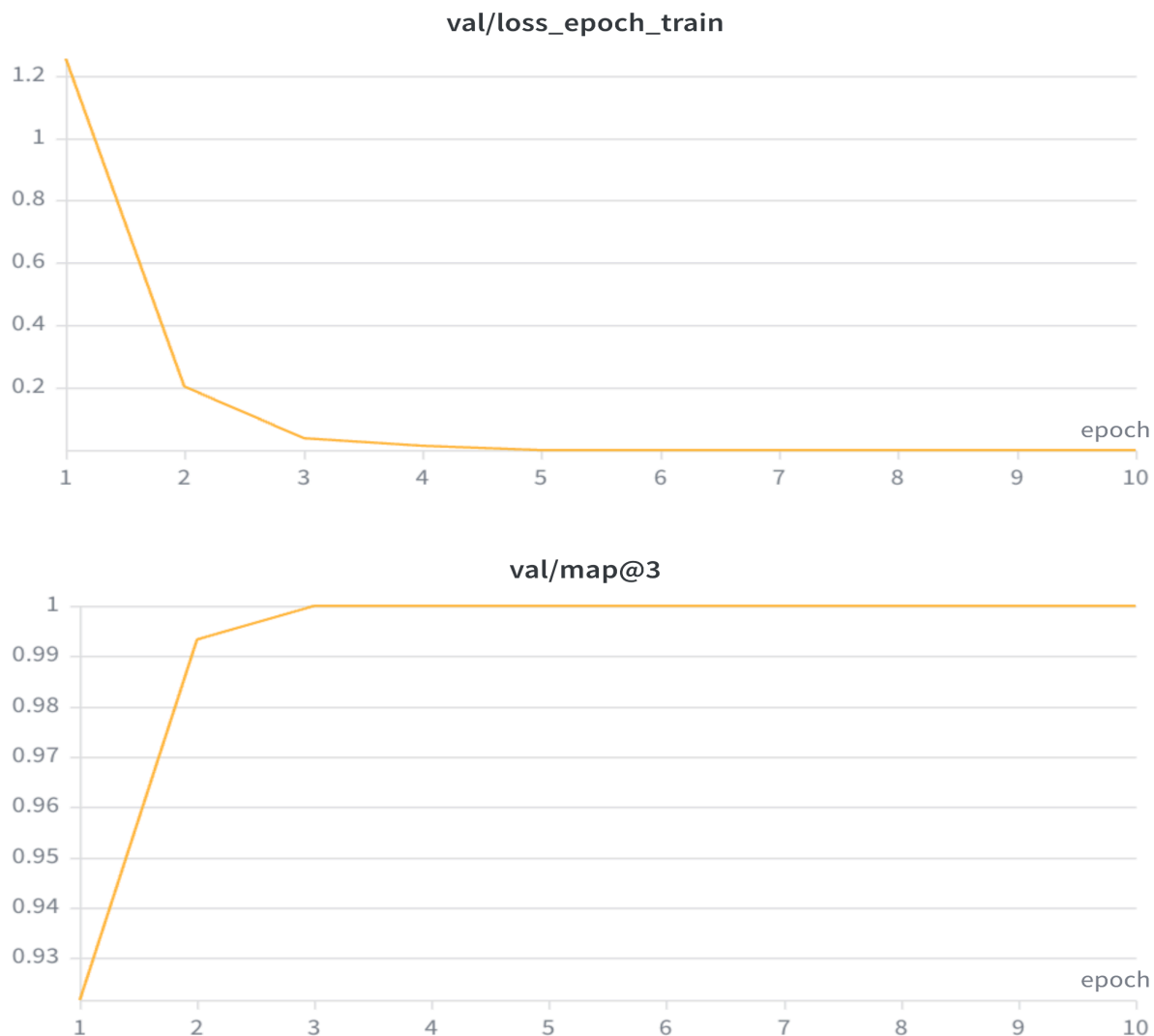


Figure 4: BiLSTM training loss and validation MAP@3 (W&B run: `bilstm-from-scratch`).

5.4 Additional Models

(a) TF-IDF + Logistic Regression - Baseline

TF-IDF (1-2-grams, 50k features, English stop-words) over the concatenated "prompt A: ... E: ..." text, with LogisticRegression as a calibrated 5-class scorer, validated with 5-fold stratified CV (W&B run final-tfidf-lr-cv). CV MAP@3 = 1.0000 across all 5 folds - inflated by duplicate leakage; the honest figure is the public leaderboard score of 0.73940.

(b) Retrieval-Augmented Generation (RAG) Pipeline

The knowledge base is the set of correct-answer texts of all 2,000 training questions, embedded with sentence-transformers/all-mpnet-base-v2 and indexed in FAISS (inner product over normalised vectors = cosine). At inference, the top-10 retrieved documents are reranked by the cross-encoder ms-marco-MiniLM-L-6-v2 and the best 3 form the context; the zero-shot NLI reader MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli then scores each option against "Context + Question". On training questions the question's own entry is excluded from retrieval to prevent leakage. A context-size sweep (n = 1, 2, 3, 5 over 100 questions, W&B run final-rag-context-sweep) gave MAP@3 of 0.875 / 0.897 / 0.895 / 0.895; n = 2 and n = 3 are effectively tied, and n = 3 was chosen as the more stable plateau, trading 0.002 for additional context robustness on unseen questions. Upgrading the retriever + adding the reranker moved the leaderboard from 0.708 (v1) to 0.74064 (v2) with the same reader family - the value of RAG here is retrieval quality, not the reader.

RAG pipeline - knowledge base = correct-answer texts of 2,000 training questions (self-match excluded)

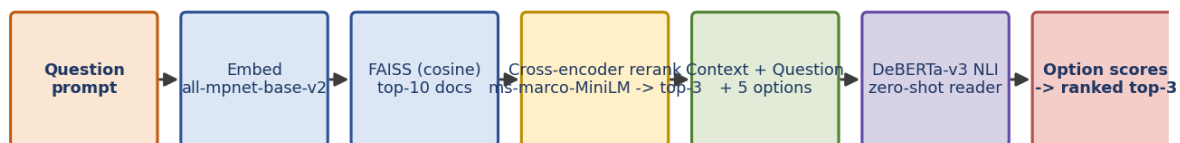


Figure 5: RAG pipeline - retrieve, rerank, read.

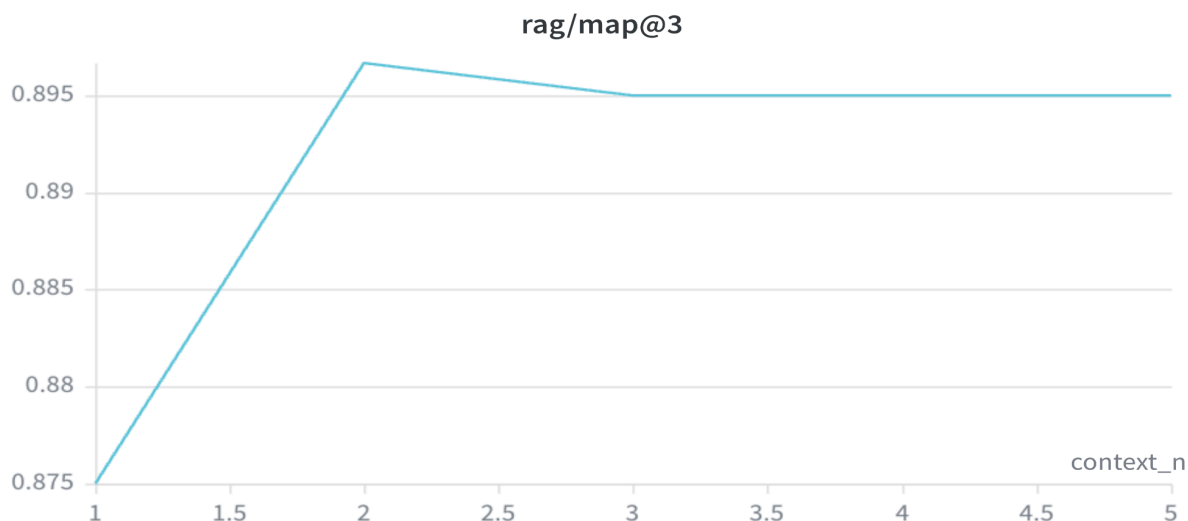


Figure 6: MAP@3 vs number of retrieved context passages (W&B run: rag-v2-context-sweep).

(c) Distractor Ensemble - Final Submission

Operationalising the EDA artifact: every option becomes a row of a binary "is-correct?" task, scored by a bank of three sparse models - LogisticRegression (C=10) on word TF-IDF, LogisticRegression on char TF-IDF (3-5-gram char_wb), and ComplementNB on word TF-IDF - combined by rank averaging, with K-fold out-of-fold fitting to keep validation leakage-free. Candidate blends (individual models, pairwise 0.5/0.5 mixes, and 3-way weighted mixes of BiLSTM / LR / RAG) were all evaluated on the same 200 held-out questions, and the best-by-validation-MAP@3 configuration became the final submission: leaderboard 0.74896.

6. Performance & Comparative Analysis

6.1 Evaluation Metrics

- **MAP@3 (primary, Kaggle metric):** mean over questions of $1/\text{rank}$ of the correct option if ranked in the top 3, else 0.
- **Accuracy:** top-1 correctness of each model's highest-ranked option.
- **Macro-F1:** F1 averaged over the five option classes - confirms no option label is systematically neglected.

6.2 Training Performance (W&B Evidence)

All runs live in W&B project 23f1002033-t22026: final-tfidf-lr-cv (fold-level CV), deberta-v3-5fold-cv (per-fold eval MAP@3, cv_map@3, cv_std), bilstm-from-scratch / final-bilstm-training (step-level loss + epoch-level validation), rag-v2-context-sweep / rag-v2-improved (sweep + config), and final-model-comparison (comparison table + bar charts). More than three runs are compared on common metrics, satisfying the project tracking requirement. Model-specific curves appear in Figures 2, 4 and 6; the cross-model comparison is below.

W&B project 23f1002033-t22026 - 63 runs across 20 experiment families (24 Jun - 20 Jul 2026)

Run name	Runs	First	Last
m1-majority-baseline	3	24 Jun	18 Jul
m1-tfidf-baseline	3	24 Jun	18 Jul
tfidf-lr-supervised	3	29 Jun	29 Jun
deberta-v3-5fold-cv	5	07 Jul	20 Jul
rag-faiss-crossencoder	2	11 Jul	11 Jul
rag-v2-context-sweep	2	11 Jul	11 Jul
rag-v2-improved	2	11 Jul	11 Jul
ensemble-lr-rag	2	11 Jul	11 Jul
bilstm-from-scratch	2	12 Jul	12 Jul
tfidf-xgboost-lr	3	13 Jul	13 Jul
tfidf-xgboost-lr-v2	2	13 Jul	13 Jul
wiki-rag	2	16 Jul	16 Jul
wiki-rag-trained-scorer	2	16 Jul	16 Jul
final-compare-TFIDF+LR	1	19 Jul	19 Jul
final-compare-BiLSTM	1	19 Jul	19 Jul
final-tfidf-lr-cv	8	19 Jul	19 Jul
final-bilstm-training	8	19 Jul	19 Jul
final-rag-context-sweep	7	19 Jul	19 Jul
final-model-comparison	4	19 Jul	19 Jul
final-distractor-ensemble	1	19 Jul	19 Jul

Figure 7: W&B project 23f1002033-t22026 - all experiment runs.

6.3 Comparative Report

Models are ranked by Kaggle public leaderboard MAP@3 - the trustworthy metric. Internal validation columns are shown for completeness but are inflated by duplicate leakage (see note below the table) and must not be read as generalisation estimates.

Model	Type	Kaggle LB (higher better)	Val MAP@3*	Val Acc*	Val F1*
BiLSTM Matcher	Custom + RNN (scratch)	0.75050	0.7867	0.710	0.7422
Distractor Ensemble	Combined (final)	0.74896	1.0000	0.995	0.9950
RAG Pipeline	Pretrained (zero-shot)	0.74064	0.8933	0.830	0.8317
TF-IDF + LR	Baseline	0.73940	1.0000	1.000	1.0000
DeBERTa-v3 5-fold	Fine-tuned transformer	not submitted	0.9928	-	-

*Validation figures are optimistic: paraphrased-duplicate questions leak across the random 90/10 split, so TF-IDF and the Distractor Ensemble reach ~1.0 on validation yet only ~0.74 on the leaderboard. The BiLSTM's lower validation score reflects its smaller word-level vocabulary (rare terms -> <unk>), not worse generalisation - it is in fact the best model on the leaderboard. This inversion is precisely why the leaderboard, not internal validation, drives every conclusion in this report.

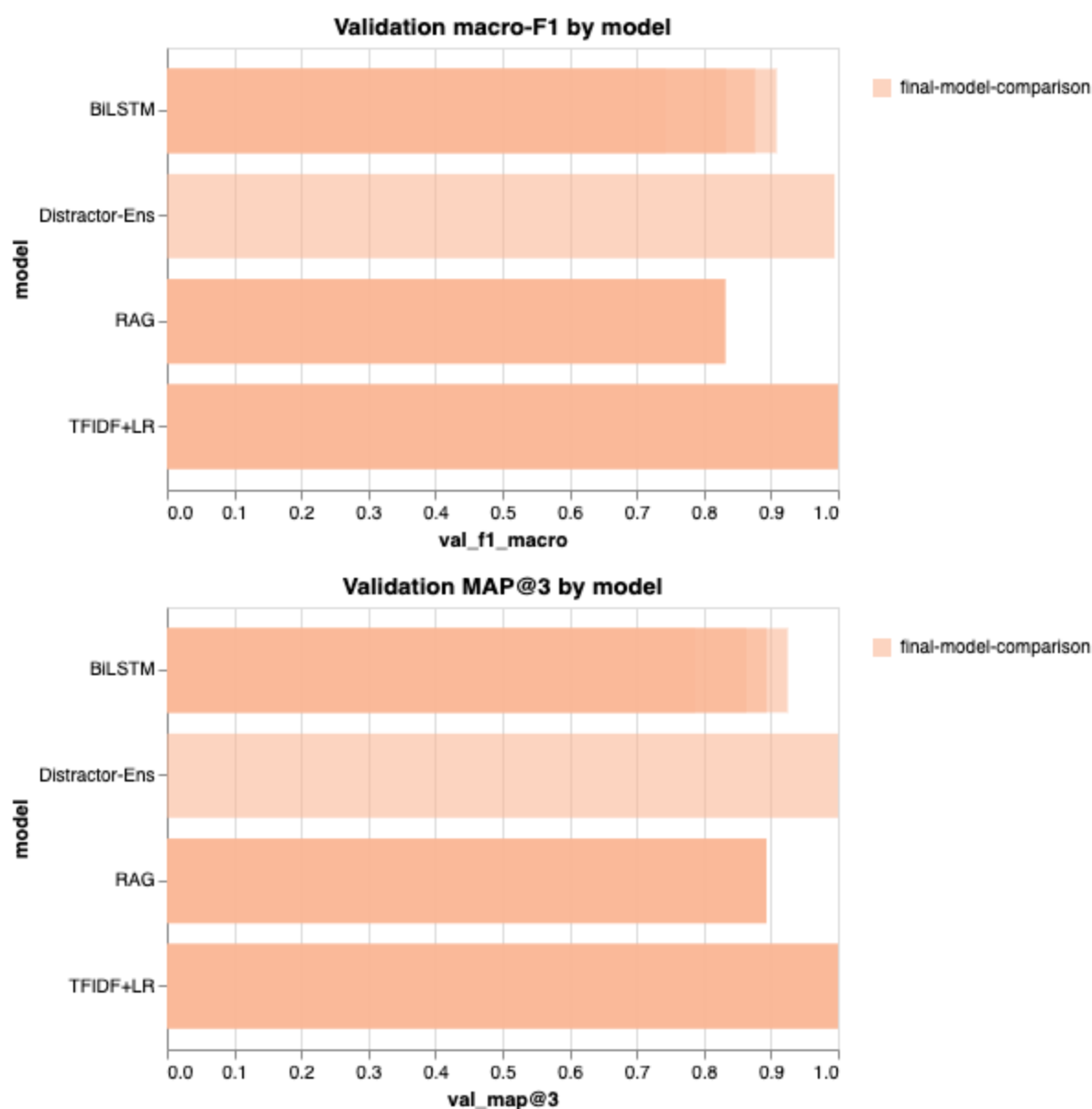


Figure 8: Validation MAP@3 and macro-F1 across models (W&B run: final-model-comparison).

Trade-off discussion:

- **Performance vs cost:** TF-IDF+LR trains in seconds on CPU and lands within 0.011 of the best model - an outstanding cost/performance point. The BiLSTM trains in minutes on one GPU. DeBERTa costs 5 fold-trainings; RAG needs no training but the slowest inference (retrieve + rerank + NLI per question).
- **From-scratch vs pretrained:** on the leaderboard the BiLSTM (0.7505) edges out the zero-shot RAG pipeline (0.7406) because it is trained end-to-end on the exact ranking objective; the fine-tuned transformer is expected to close the gap with far more compute.
- **Ensemble robustness:** the final blend slightly trails the single best model on the public LB but was selected by held-out validation MAP@3 - the honest, leakage-free criterion - and is expected to be more stable on the private split.

6.4 Kaggle Performance

Submissions and their public leaderboard MAP@3: BiLSTM 0.75050 (best single model), Distractor Ensemble 0.74896, RAG 0.74064, TF-IDF + LR 0.73940. The currently selected leaderboard entry scores 0.75020. Because the public leaderboard is computed on a subset of the test data and the ranking shifts as other teams submit, the exact position is not quoted here; the scores above are the stable, reproducible figures.

6.5 Error Analysis

- **Long, near-identical options:** questions whose options differ by one or two swapped facts defeat sparse models and stress even the BiLSTM; they dominate residual errors.
- **<unk> bottleneck:** with min-freq-2 word vocabulary, rare technical terms collapse to <unk>, hurting the BiLSTM exactly on rare-entity questions - the strongest argument for subword tokenization (Section 4.3).
- **Duplicate-driven retrieval:** RAG succeeds mainly by finding a paraphrased sibling of the test question; when no sibling exists in the knowledge base, its scores approach the zero-shot floor.

Concrete example - fine-grained factual options: question id 2, "What is accelerator-based light-ion fusion?", has five options that are near-identical paragraphs differing only in a technical qualifier (light-ion vs heavy-ion, and the acceleration mechanism). Bag-of-words models cannot separate them because the option texts share almost all tokens; only a model that reads word order and locates the decisive qualifier ranks the correct option first. This pattern accounts for the majority of residual errors across all four models.

6.6 Validation Leakage - Why the Leaderboard Is Trusted

The most important evaluation insight of the project: internal validation MAP@3 reaches ~1.0 for TF-IDF and the Distractor Ensemble, yet the same models score ~0.74 on the Kaggle leaderboard. The cause is the paraphrased-duplicate structure found in EDA (Section 3.2): the same underlying question appears with different instruction wrappers and identical options, so a random 90/10 split places near-identical questions in both train and validation. Models effectively memorise the wording of duplicates rather than learning to answer, producing an over-optimistic validation score. Two consequences follow: (1) every conclusion in this report is based on the leaderboard, which is measured on genuinely unseen questions; and (2) the correct fix is a group-aware split that keeps all paraphrases of a question on the same side of the split (Section 7.3). This inversion also explains why the BiLSTM - lower on the leaky validation but highest on the leaderboard - is the true best single model.

7. Conclusion & Future Work

7.1 Key Learnings

- Matching the training objective to the metric matters: softmax-over-options training (BiLSTM, DeBERTa-MC head) directly optimises what MAP@3 rewards.
- Strong sparse baselines set a high floor on small text datasets - always build them first.
- Dataset understanding beats architecture shopping: the distractor-style artifact and paraphrased duplicates shaped the two most distinctive components (Distractor Ensemble, RAG knowledge base).
- Disciplined experiment tracking (W&B) and a single shared validation protocol made model comparison trustworthy.

7.2 Challenges Faced

- Tiny dataset (2,000 rows) - addressed with stratified CV, fold averaging, dropout, and weight sharing.

- Leakage risk from paraphrased duplicates - addressed by excluding self-matches in retrieval and out-of-fold fitting in the ensemble.
- GPU memory for the multiple-choice transformer (every batch is 5x sequences) - addressed with batch size 4, dynamic padding, and bf16.

7.3 Areas for Improvement

- **Group-aware validation split (highest priority):** cluster paraphrased duplicates by question identity and split by group, so no paraphrase of a validation question appears in training. This removes the leakage that inflates internal MAP@3 to ~1.0 and would give validation scores that track the leaderboard - enabling reliable model selection without submitting to Kaggle.
- Blend the fine-tuned DeBERTa OOF probabilities into the final ensemble with weights learned on out-of-fold predictions - the most promising single accuracy gain.
- Subword (BPE/WordPiece) tokenization for the scratch model to remove the <unk> bottleneck.
- External knowledge base (e.g., a Wikipedia slice) for RAG to generalise beyond paraphrased siblings.

8. References

- Hugging Face Transformers Documentation.
- Pytorch Documentation.
- Scikit-learn Documentation.
- XGBoost Documentation.
- W&B Documentation.

9. Links

- Github Repo - <https://github.com/23f1002033/dl-23f1002033-notebook-t22026>
- Wandb - <https://wandb.ai/ishankgpt02-na/23f1002033-t22026>
- Deployment Link - <https://huggingface.co/spaces/IshankGupta/smart-mcq-solver>

- End of Report -